Web Service 与第一版管理平台：实现与代码讲解

到上一节为止，底座的能力全部就位：会调模型、会思考、有入口、能推送、能干活、记得住、有安全边界、还能到点自动跑。但这些能力目前只有 CLI 一个对外出口——业务系统没法用。这节补上能力五：Web Service 把内部能力包装成 REST API，顺带做出第一版管理平台。照旧四件事：是什么、动手前想清楚什么、代码怎么写、做完怎么验。

技术栈还是 JDK 21 + Spring Boot 3.x，HTTP 层用 Spring MVC + virtual thread。下面的代码是示意。

一、Web Service 是什么，干嘛用的

一句话：**Web Service 是 OryxOS 的对外门面——没有它，OryxOS 只是一个 CLI 工具；有了它，任何能发 HTTP 请求的业务系统都能把 Agent 接进自己的流程。**

18 节讲 CLI 时埋过一句：`oryxos serve` 启动 Web Service，26 节细讲——就是现在。`serve` 起来之后，Spring MVC 监听 8080 端口，对外暴露 10 个端点，统一前缀 `/api/v1`，按资源分成六个 Controller：



每个 Controller 只做参数校验、响应包装、错误处理——实际逻辑全部委托给核心层，跟 CLI 走同一个 AgentService

这 10 个端点覆盖四类事：**会话管理**（创建、发消息、查历史、归档，4 个）、**Agent 调用**（无状态 invoke，1 个）、**信息查询**（Profile / Memory / Tool，3 个）、**系统状态**（health / info，2 个）。业务系统最常用的两条路：要连续对话就先创建 Session 再多次发消息；要一次性调用就直接 `POST /agents/{name}/invoke`。

管理平台是这节的第二个交付物：一个跑在 `/admin` 的静态前端，调上面这 10 个端点，把会话、Profile、Tool、记忆、状态渲染成能看的页面。它没有自己的后端——所有数据都来自这 10 个端点，这也顺带验证了"API 是完备的对外通道"这件事本身。

二、动手前先想清楚几件事

第一，Controller 必须薄，跟 CLI 是同一个引擎。 18 节讲过 CLI 是"读—转交—打印的壳"，Web Service 的 Controller 是一模一样的角色：参数校验、响应包装、错误处理，三件事之外一概不碰。`POST /sessions/{id}/messages` 进来后调的还是那个 `AgentService.process`——跟 `oryxos chat` 走的是同一个方法。人推的两个入口共享同一个引擎，这是架构上反复强调的点，写代码时最容易被"顺手在 Controller 里加点逻辑"破坏掉。

第二，异常出口只有一个。 六个 Controller 都不自己拼错误响应，所有异常统一交给 `GlobalExceptionHandler` 转成标准 JSON（`errorCode`、`message`、`timestamp`）。错误码用标准 HTTP 语义：400 参数错误、404 资源不存在、500 内部错误、503 Provider 故障；Agent 调用最长 60 秒，超时返回 504。这个口径一开始就定死，业务系统对接时才不用逐个端点猜错误格式。

第三，核心阶段明确不做的，列出来别手痒。 认证（假设内网部署，扩展阶段补 API Key + JWT）、SSE 流式响应、WebSocket、限流、RBAC——全部不做。请求大小限制倒是要有：单条消息最大 32KB，Session 历史返回最多最近 100 条，这两条是防呆不是治理。

第四，`serve` 要能只靠一个 provider key 就起得来（坑：Spring AI eager 装配）。 这是本节第一次真正让人跑 `serve` 常驻，一个启动坑会集中暴露：deepseek 走的是 spring-ai-openai starter，它的 `OpenAiAutoConfiguration` 会**急切实例化**一个 `OpenAiChatModel` Bean，启动即索要 `spring.ai.openai.api-key`——哪怕 OryxOS 根本不用这个自动 Bean（Provider 由 `ProviderChatModelFactory` 按宪法 III 显式构造）。不处理的话，`serve` 会因为"缺 spring.ai.openai.api-key"直接启动失败，逼运营方配两个 key，还会卡死 31 节"干净机器 30 分钟部署"。解法跟 16 节排除 `DashScopeAutoConfiguration` 是同一招（宪法 II：禁用 Spring AI 的 eager 模型自动装配）——在 `application.yaml` 的 `spring.autoconfigure.exclude` 里把 `OpenAiAutoConfiguration` 也排掉，让 `serve` 只认 `DEEPSEEK_API_KEY` 一个环境变量。

第五，管理平台的边界：这一版只读。 能看会话、看 Profile、看 Tool、看记忆、看状态，但**不能**创建或修改 Agent——"通过 API 定义一个 Agent"需要的那套端点（29、30 节）现在还不存在，界面上也不该出现假按钮。另外这个前端本身是一次"用提示词做前端"的实战：把页面需求描述清楚交给 AI 生成 Vue 页面（跟 `website/` 首页同栈 Vue 3 + Vite），我们只验收不手写。

想清楚就这几句：Controller 薄、跟 CLI 共享引擎；异常单出口、错误码定死；不做认证流式限流；管理平台这版只读。

三、代码怎么写

第一步：把 virtual thread 打开。 `serve` 命令启动 Spring 上下文（18 节的"重命令"），`application.yaml` 里一行配置让每个请求跑在独立虚拟线程上：

```yaml
spring:
  threads:
    virtual:
      enabled: true
  # 宪法 II：禁用 Spring AI 的 eager 模型自动装配。deepseek 走 spring-ai-openai
starter，
  # OpenAiAutoConfiguration 会急切实例化 OpenAiChatModel 并索要
spring.ai.openai.api-key；
  # OryxOS 不用这个自动 Bean（Provider 由 ProviderChatModelFactory 显式构造），排
掉后 serve 只需 DEEPSEEK_API_KEY。
  autoconfigure:
    exclude:
      -
com.alibaba.cloud.ai.autoconfigure.dashscope.DashScopeAutoConfiguration
      - org.springframework.ai.autoconfigure.openai.OpenAiAutoConfiguration
server:
  port: 8080
```

同步阻塞的代码直进直出，虚拟线程负责在 LLM 调用这种 IO 等待时自动让出——不引入 WebFlux、不写一行响应式代码，这是决策三、决策五定好的。两个 `autoconfigure.exclude` 是决策四的落地：不排掉 `OpenAiAutoConfiguration`，`serve` 会因缺 `spring.ai.openai.api-key` 启动失败（排除类的确切全限定名以 `mvn dependency:tree` 里锁定的 spring-ai 版本为准）。

第二步：最典型的 Controller。 拿承载 ReAct 循环的那个端点举例：

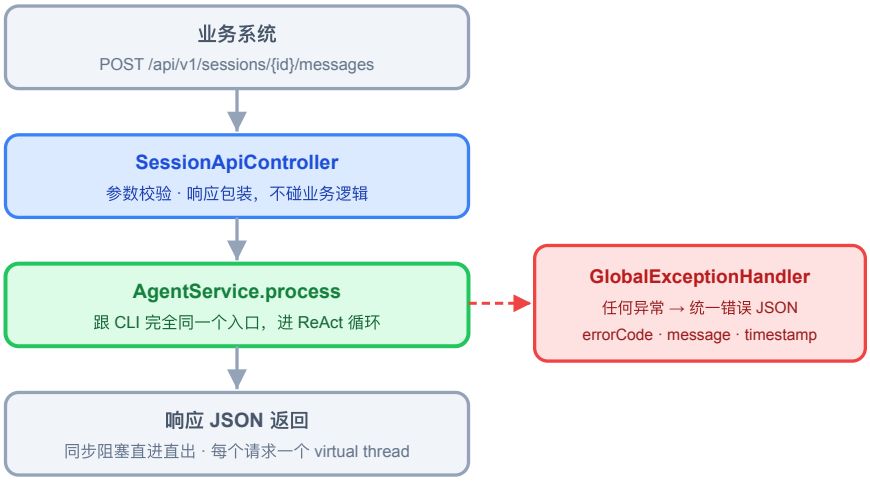
```java
@RestController
@RequestMapping("/api/v1/sessions")
public class SessionApiController {

    private final AgentService agentService;
    private final SessionManager sessionManager;

    @PostMapping("/{id}/messages")
    public MessageResponse send(@PathVariable String id, @RequestBody
MessageRequest req) {
        if (req.content() == null || req.content().length() > 32 * 1024) {
            throw new InvalidRequestException("消息为空或超过 32KB");   // →
400
        }
        Session session = sessionManager.get(id)
                .orElseThrow(() -> new SessionNotFoundException(id));   // →
404
        String reply = agentService.process(session, req.content());   // 跟
```

```
CLI 同一个入口
        return new MessageResponse(reply);
    }
}
```

一次请求从进到出的完整链路，以及异常怎么被统一接走：



400 参数错 · 404 不存在 · 500 内部错 · 503 Provider 故障 · Agent 调用超 60 秒返回 504

第三步：统一异常出口。

统一信封用既有的 `ApiResponse`（`code` / `message` / `data` / `timestamp`，第24节随白名单端点已建于 `oryxos-web`）——本节复用、不另建 `ErrorBody`；`GlobalExceptionHandler` 在既有类上**扩展**新的异常映射：

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 404 —— 领域资源不存在（会话 / Agent）
    @ExceptionHandler({SessionNotFoundException.class,
ResourceNotFoundException.class})
    public ResponseEntity<ApiResponse<Void>> notFound(RuntimeException e) {
        return ResponseEntity.status(404).body(ApiResponse.error(404,
e.getMessage()));
    }

    // 503 —— Provider 故障
    @ExceptionHandler({IllegalStateException.class,
ProviderUnavailableException.class})
    public ResponseEntity<ApiResponse<Void>> providerDown(RuntimeException
e) {
        return ResponseEntity.status(503).body(ApiResponse.error(503,
e.getMessage()));
```

```
    }

    // 504 —— Agent 调用超时（60s 上限）
    @ExceptionHandler(AgentTimeoutException.class)
    public ResponseEntity<ApiResponse<Void>> timeout(AgentTimeoutException
e) {
        return ResponseEntity.status(504).body(ApiResponse.error(504,
e.getMessage()));
    }

    @ExceptionHandler(Exception.class)   // 兜底
    public ResponseEntity<ApiResponse<Void>> internal(Exception e) {
        return ResponseEntity.status(500).body(ApiResponse.error(500,
"Internal server error"));
    }
}
```

注意兜底那条不把 `e.getMessage()` 原样吐给外部——内部异常细节进日志，对外只说"内部错误"，这是门面该有的分寸。（既有的 `IllegalArgumentException→400`、`NoResourceFoundException→404` 映射保留不动。）

第四步：其余端点都很直。 `POST /agents/{name}/invoke` 临时建一个一次性 Session、跑完返回；`GET /profiles`、`GET /tools` 分别列 `ProfileRegistry` 和 `ToolRegistry` 的内容；`GET /memory` 返回 `MEMORY.md` 全文；`GET /health` 回一个 ok；`GET /info` 多带一份各 Provider 的连通状态。OpenAPI 文档用 `springdoc-openapi` 自动生成，暴露在 `/swagger-ui`，不用手写接口文档。

第五步：管理平台，用提示词做出来（Vue，跟官网首页同栈）。 前端不手写，给 AI 一段这样的提示词——技术栈跟 `website/` 首页保持一致（Vue 3 + Vite），别用裸 HTML，视觉和工程习惯统一：

```
做一个 Vue 3 + Vite 的单页管理界面（跟项目 website/ 首页同栈、同风格）：
左侧导航五项：会话列表、Profile 列表、Tool 列表、长期记忆、运行状态；
分别调 GET /api/v1/sessions、/profiles、/tools、/memory、/info 渲染成表格或文本；
只读，不要任何新建/编辑/删除按钮；出错时把统一错误 JSON 里的 message 显示出来；
vite base 配成 '/admin/'，build 产物用相对资源路径，好让 Spring 托在 /admin 子路径
下。
```

前端源码放 `oryxos-web/src/main/frontend/`（Vue+Vite 工程），`npm run build` 产出到 `oryxos-web/src/main/resources/static/admin/`（`vite.config` 的 `base: '/admin/'` + `build.outDir` 指过去），由 Spring 直接托管——`serve` 起来后访问 `http://localhost:8080/admin` 就能看。构建串联有两种做法，plan 里二选一说明：用 `frontend-maven-plugin` 把 `npm ci && npm run build` 绑进 `mvn package`（一条命令出全

量 fat JAR），或前端单独 `npm run build` 后再 `mvn package`（构建更快、CI 更简单）。**注意 SPA 的前端路由**：Spring 要对 `/admin/**` 的未命中路径回落到 `admin/index.html`（`GET /api/v1/**` 不受影响），否则刷新子路由会 404。

页面风格：钉死到官网首页的设计语言，不引外部 skill。 页面好不好看不靠碰运气，靠把风格写进提示词、且钉到已有的一套设计 token 上——`website/` 首页那套（见 `website/.vitepress/theme/custom.css`）就是唯一风格来源。生成提示词里追加这段视觉约束（值直接抄首页 token，一个字别自创）：

```
视觉风格（与 OryxOS 官网首页完全一致，值取自 website/.vitepress/theme/custom.css）：
- 深色主题：背景 #000000 / 卡片 #111111 / 悬浮块 #1a1a1a；分隔线与边框 #222222；
- 主色（橙）：#f97316 为主、hover/强调用 #ea6a00 / #c2550a；主色仅用于强调（激活项、链接、数值高亮），不铺大面积；
- 文字：主 #f5f5f5 / 次 #a3a3a3 / 弱 #666666；
- 字体：正文 Inter，代码/ID/JSON 用 JetBrains Mono（等宽）；
- 左侧竖直导航（深色）+ 右侧内容区；表格深色、行分隔用 #222；状态用小圆点/标签（成功绿、失败红、警告橙）；
- 顶部放 /logo.svg + "OryxOS 管理台"；整体克制、留白足、圆角小（4–6px），跟首页一个气质；
- 响应式，窄屏导航收起；空数据/加载中/错误三态都要有明确占位，别白屏。
```

把这套风格固化成一个项目内 skill：`oryxos-admin-ui`。 上面的 token 和组件规范不要每次手抄进提示词——管理台这一版要建、30 节还要加"Agent 管理"页，同一套风格会用不止一次。所以本节顺手把它沉淀成一个**项目内** skill `.claude/skills/oryxos-admin-ui/SKILL.md`：里面装着这套设计 token（深色 + 橙、Inter/JetBrains Mono）、工程约定（`base:'/admin/'`、产物落 `static/admin`、SPA 回落、只调 `/api/v1`）、布局/三态/响应式规范和验收清单。之后无论"生成整套只读管理台"还是"给管理台加一页"，都**调这个 skill**，产出天然跟首页同源、可复现。

> **为什么是项目内 skill，而不是引外部 skill（如 `mattpocock/skills`、superskill）？** 我核过 `mattpocock/skills` 的内容——它是"给工程师的工作流 skills"（PRD、TDD、重构、git 安全那类**流程**技能），不是 UI/设计技能，套到"管理台长什么样"上牛头不对马嘴；外部通用件也带不来"跟 OryxOS 首页一致"这件事。真正决定一致性的是**项目自己的设计 token**，所以把它做成**项目内**、直接引用首页 token 的 skill，而不是外挂一个跟本项目审美无关的第三方件。确需组件库时也只选可被 token 完全覆盖的 headless 方案。



第一版只能"看"：列表、详情、状态。"能管"（通过 API 新建 / 编辑 / 删除 Agent）要等 30 节补上

有几样先别做。 Profile 的增删改端点、Memory 的写入端点、Webhook 触发、Prometheus metrics——这些都在扩展阶段清单里；其中"通过 API 定义 Agent"这一块（`/api/v1/agents` 的增删改）是 29、30 节的正题，这节刻意留白。

本节交付物（Spec-Kit 拆解锚点）：

- 代码：六个 Controller（Session/Agent/Profile/Memory/Tool/System）、`GlobalExceptionHandler`（扩展既有、复用 `ApiResponse` 信封）；`springdoc-openapi` 集成
- 测试：`SessionApiControllerTest`、`GlobalExceptionHandlerTest`、`WebSmokeIT`（见验收 harness）
- 配置：`spring.threads.virtual.enabled=true`、端口 8080、消息 32KB / 历史 100 条限制
- 前端：`oryxos-web/src/main/frontend/` 下 AI 生成的 Vue 3 + Vite 只读管理台（五页调五个 GET 端点），`npm run build` 产出到 `static/admin/`、Spring 托在 `/admin`；构建串联方式（frontend-maven-plugin 绑进 mvn / 手动 npm build）plan 里二选一，SPA 路径回落 `/admin/**` → `index.html`
- 风格 skill：`.claude/skills/oryxos-admin-ui/SKILL.md`（固化首页设计 token + 工程约定 + 组件/三态规范 + 验收清单）——生成管理台就调它，30 节加 Agent 管理页复用同一套

四、验收 harness：把验收标准变成可执行的测试

Web 层的 harness 用 `@WebMvcTest` 切片——只起 MVC 层、mock 掉 `AgentService`，不碰模型不碰库，跑得飞快：

测试类	覆盖的验收点
`SessionApiControllerTest`	超 32KB → 400；Session 不存在 → 404；正常请求 `agentService.process` 恰被调一次（Controller 没夹

测试类	覆盖的验收点
	带私货）
`GlobalExceptionHandlerTest`	每类异常映射到约定状态码；响应体都是统一 `ApiResponse` 信封；**500 时响应里不含内部异常的 message**（不泄漏）
`WebSmokeIT`（`@SpringBootTest` 起真实上下文，不依赖模型）	`/health`、`/info`、`/profiles`、`/tools` 真实链路可达——验证 Bean 装配和扫描范围没炸

最值钱的一个——"门面的分寸"回归：

```
@Test
void 内部异常细节_绝不能出现在500响应里() {
    when(agentService.process(any(), any()))
        .thenThrow(new IllegalStateException("jdbc:sqlite:/data/oryxos.db
connect failed"));

    mockMvc.perform(post("/api/v1/sessions/s-
1/messages").contentType(JSON).content(body))
        .andExpect(status().isInternalServerError())
        .andExpect(jsonPath("$.errorCode").value("INTERNAL_ERROR"))
        .andExpect(jsonPath("$.message").value("内部错误"))
// 统一话术
        .andExpect(content().string(not(containsString("jdbc:sqlite"))));
// 连接串这类内幕一个字不漏
}
```

`WebSmokeIT` 有一个特别的价值：它会真实触发 JPA repository 扫描——18 节那个 "Found 0 repositories" 的坑如果在 web 模块复发，这里第一时间红，不用等到手动 `serve`。

五、怎么用，做完怎么验

```
oryxos serve                                # 启动，默认 8080
curl -X POST localhost:8080/api/v1/sessions          # 建会话
curl -X POST localhost:8080/api/v1/sessions/{id}/messages \
     -H 'Content-Type: application/json' -d '{"content":"今天北京天气怎么样"}'
curl localhost:8080/api/v1/tools                     # 列工具
open http://localhost:8080/admin                     # 管理平台
open http://localhost:8080/swagger-ui                # 接口文档
```

harness 全绿后，剩下的人工确认：

- 10 个端点 curl 真链路全走通（含真模型的 `POST /sessions/{id}/messages`），审计有账。
- CLI 聊过的 Session 从 `GET /sessions/{id}` 能查到——两个人推入口共享存储。
- 断掉 Provider 拿 503、构造超 60 秒调用拿 504（这两个依赖真实故障注入，切片测试模拟不了完整链路）。
- 并发压一把（200 并发 invoke），虚拟线程扛得住。
- 管理平台五个页面渲染真实数据、无写操作入口；`/swagger-ui` 文档齐全。
- 400/404 映射、错误格式统一、500 不泄漏——已由 harness 覆盖，`mvn test` 绿即打勾。

到这一步，底座的五大核心能力全部有了对外出口。但现在的 Agent 还是散装的——模块都在，整条链路没有从头到尾拉通过。接下来两节就干这件事：串联。